SECTION 1

Recurrent Neurons and Layers

- **Recurrence:** Outputs depend on current inputs and prior hidden state; unrolling through time exposes a deep chain sharing weights `Wx` and `Wy` across steps.
- **Weight tying:** The same parameters at every time step are what make an RNN compact but also create long backprop chains through time.
- **Sequence modes:** Nets may map sequence-to-sequence, sequence-to-vector (many inputs, one output), vector-to-sequence, or encoder–decoder pairs for tasks like translation where the full input must be seen before decoding.
- **Dense shortcuts:** Very short sequences can still be handled by Flatten plus dense layers, but recurrence or convolution wins as length grows.
- **Memory cells:** The hidden state carries short-term context; vanilla cells remember only roughly ten steps unless augmented.

SECTION 2

Training RNNs

- **BPTT:** Training unrolls the net over time and applies standard backprop; gradients flow through all time steps used in the loss and sum over shared weights across steps.
- **Keras:** `SimpleRNN`, `LSTM`, `GRU`, and `RNN(custom_cell)` hide tape logic; `return_sequences` controls whether each step or only the last output is returned.
- **Custom cells:** Subclassing `Layer` with `call(inputs, states)` enables LayerNorm or other per-step transforms inside the recurrence.

SECTION 3

Forecasting a Time Series

- **Setup:** Univariate series use tensors shaped `[batch, time steps, features]`; establish naive and linear baselines before trusting RNNs.
- **Preprocessing:** Remove trend and seasonality before classical ARIMA-style comparison when required; RNNs sometimes learn trend directly but differencing can still help.
- **Depth:** Stacked `SimpleRNN` layers can underperform a linear head on smooth synthetic waves; deeper stacks with a final `Dense` output often win; `TimeDistributed(Dense)` trains multi-step forecasts at every timestep for richer gradient flow.
- **Deep RNNs:** Multiple recurrent layers need `return_sequences=True` on all but the last recurrent block so each layer receives a full 3D sequence.
- **Horizons:** Predict next step, fixed vector of future steps, or autoregressive rollouts by feeding predictions back—error compounds on long rollouts but suits generation-style use cases.
- **Baselines:** Compare against naive last-value and flattened linear models so RNN gains reflect real temporal structure rather than implementation effort.
- **Uncertainty:** MC Dropout inside recurrence can provide spread estimates on future points when probabilistic forecasts matter.

SECTION 4

Handling Long Sequences

- **Unstable gradients:** Deep unrolls echo Chapter 11; remedies include gradient clipping, dropout and recurrent_dropout, Layer Normalization in custom cells, and saturating activations (default tanh in simple RNNs).
- **Batch norm caveat:** BatchNormalization inside the time axis rarely helps; normalizing between stacked recurrent layers is the more typical pattern.
- **Short-term memory limit:** Vanilla RNNs lose early inputs on long sequences; LSTM splits cell and hidden states with forget, input, and output gates to store and retrieve information over many steps; GRU merges gates for a lighter variant with similar practical performance.
- **Peephole variant:** `PeepholeLSTMCell` feeds cell state into gate computations and can help on some tasks though it is not the default stack.
- **Conv1d hybrid:** Strided convolutions shorten sequences before RNN layers so higher layers see longer effective history with fewer steps.
- **WaveNet:** A stack of dilated causal `Conv1D` layers expands receptive field exponentially without recurrence, handling very long sequences (e.g., raw audio) and strong multi-step forecasts on the chapter's toy series when trained as a fully convolutional forecaster.
- **Padding:** Causal convolutions use `padding="causal"` so each step only sees past inputs, matching real-time forecasting constraints.
- **CNN on sequences:** For long 1D signals, pure convolution (sometimes with stacked dilations) can match or beat RNNs on accuracy while parallelizing better across time steps.
- **Chapter arc:** NLP-heavy sequence modeling with attention moves to Chapter 16, but the forecasting and memory foundations here underpin those architectures.
- **Causal modeling:** Forecasting trainers rely on causal masks so future targets never leak into past states during multi-step loss construction.